

Ниже — готовые краткие ответы по вопросам 1–15. Основные источники: лекция 01 про корпуса, языковое моделирование, n-граммы, закон Ципфа, stop words и матрицу совместной встречаемости ; лекция 02 про семантику, WordNet, TF-IDF, SVD, Word2Vec и FastText ; лекция 05 про RNN/LSTM/GRU ; лекция 04 про seq2seq, проблемы RNN и attention .

1. Корпуса текстов. Закон Ципфа и стоп-слова

Корпус текстов — это специально подготовленная коллекция текстов для изучения и моделирования языка. Корпусы нужны для статистического анализа, проверки лингвистических гипотез, обучения языковых моделей и тестирования NLP-систем.

Закон Ципфа говорит, что частота слова обратно пропорциональна его рангу в списке частотности:

$$f(r) \approx \frac{C}{r^\alpha}$$

где r — ранг слова по частоте, $f(r)$ — частота слова, C — константа, обычно $\alpha \approx 1$.

То есть самые частые слова встречаются очень часто, а большинство слов — редко. Например, служебные слова вроде «и», «в», «на», «the», «of» обычно находятся вверху частотного списка.

Стоп-слова — это очень частые слова, которые часто несут мало смысловой информации для задачи: союзы, предлоги, частицы. Их иногда удаляют при классификации, поиске или TF-IDF, чтобы не засорять признаки. Но удалять их нужно осторожно: в задачах генерации текста, перевода или анализа синтаксиса они могут быть важны.

Коротко для экзамена: корпус — база текстов для обучения и анализа; закон Ципфа объясняет, почему есть немного очень частых слов и много редких; стоп-слова — частые служебные слова, которые часто удаляют при предобработке.

2. Языковые модели, n-граммы, вероятностная модель с допущением Маркова

Языковая модель оценивает вероятность последовательности слов:

$$P(w_1, w_2, \dots, w_T)$$

Она отвечает на вопрос: насколько естественно или вероятно данное предложение в языке.

По правилу цепочки:

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1})$$

Но учитывать всю историю сложно, поэтому вводится **допущение Маркова**: вероятность следующего слова зависит только от нескольких предыдущих слов.

Для **n-граммной модели**:

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1})$$

Например, для биграмм:

$$P(w_t \mid w_{t-1})$$

Для триграмм:

$$P(w_t \mid w_{t-2}, w_{t-1})$$

Оценка вероятности по корпусу:

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

где C — количество встреч в корпусе.

Проблема: если n -грамма не встретилась в обучающем корпусе, вероятность будет нулевой. Поэтому используют сглаживание, например add-one smoothing.

Коротко для экзамена: языковая модель предсказывает вероятность текста; n -граммная модель упрощает задачу, считая, что следующее слово зависит только от последних $(n-1)$ слов.

3. Перплексия. Перекрёстная энтропия языковой модели

Перекрёстная энтропия показывает, насколько хорошо языковая модель предсказывает реальные слова в тексте. Для тестовой последовательности:

$$H = -\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid context_i)$$

Чем меньше H , тем лучше модель.

Перплексия — это экспонента от перекрёстной энтропии:

$$PP = e^H$$

Если логарифм по основанию 2:

$$PP = 2^H$$

Интуитивно перплексия показывает «среднее число вариантов», между которыми модель как будто выбирает на каждом шаге. Например, если perplexity = 50, модель в среднем как будто выбирает из 50 равновероятных слов.

Связь:

$$PP = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid context_i) \right)$$

Коротко для экзамена: перекрёстная энтропия измеряет ошибку предсказания слов, а перплексия — удобная интерпретация этой ошибки. Чем ниже perplexity, тем лучше языковая модель.

4. Семантика в NLP, семантические отношения, тезаурус. WordNet

Семантика изучает смысл языковых единиц: слов, словосочетаний, предложений и текстов. В NLP семантика важна для машинного перевода, анализа тональности, вопросно-ответных систем и поиска.

Основные проблемы семантики:

слово + контекст → значение

Например, слово «банк» может означать финансовую организацию или берег реки. Значение определяется контекстом.

Семантические отношения:

Отношение	Пример
Синонимия	автомобиль — машина
Антонимия	горячий — холодный
Гипонимия / гиперонимия	роза → цветок
Меронимия / холонимия	колесо → автомобиль

Тезаурус — словарь, который фиксирует смысловые связи между словами. Он используется для расширения поисковых запросов, анализа сходства текстов, исправления ошибок и улучшения векторных представлений.

WordNet — лексическая база данных английского языка. В ней слова организованы в **синсеты** — наборы синонимов, выражающих один концепт. WordNet содержит определения, примеры употребления, иерархию понятий и разные значения многозначных слов.

Пример: слово good может иметь несколько синсетов в зависимости от значения: «хороший по качеству», «морально правильный», «полезный» и т. д.

Коротко для экзамена: семантика отвечает за смысл, тезаурус хранит смысловые связи, WordNet — пример компьютерного тезауруса, где слова объединены в синсеты и связаны отношениями типа «часть-целое», «род-вид», «синоним».

5. Векторное представление слова, дистрибутивная семантика, матрица совместной встречаемости, bag of words

Главная идея **дистрибутивной семантики**:

слова, встречающиеся в похожих контекстах, имеют близкие значения

То есть значение слова можно понять по словам, которые часто встречаются рядом с ним.

Матрица совместной встречаемости строится так:

	context 1	context 2	context 3
word 1	5	0	2
word 2	0	7	1

Строка матрицы — это вектор слова. Если два слова имеют похожие строки, значит они встречаются в похожих контекстах и, вероятно, семантически близки.

Обычно используется окно контекста, например ± 3 слова.

Bag of Words — модель «мешка слов». Текст представляется вектором частот слов:

$$d = (c_1, c_2, \dots, c_V)$$

где V — размер словаря, c_i — сколько раз слово i встретилось в документе.

Главный минус Bag of Words: порядок слов игнорируется.

Например:

«кошка поймала мышь» «мышь поймала кошку»

Для BoW эти предложения могут быть почти одинаковыми, хотя смысл разный.

Коротко для экзамена: слова и документы можно представить векторами; BoW считает частоты слов, матрица совместной встречаемости считает соседства слов, а дистрибутивная гипотеза связывает похожий контекст с похожим смыслом.

6. TF-IDF. Векторы малой размерности. SVD

TF-IDF — способ взвешивания слов в документе. Он учитывает не только частоту слова в документе, но и то, насколько слово редкое во всей коллекции.

$$TFIDF(t, d) = TF(t, d) \cdot IDF(t)$$

где:

$$TF(t, d) = \frac{\text{количество вхождений } t \text{ в } d}{\text{общее число слов в } d}$$

$$IDF(t) = \log \frac{N}{DF(t)}$$

N — число документов, $(DF(t))$ — число документов, где встретился термин t .

Если слово часто встречается в одном документе, но редко в коллекции, его TF-IDF будет высоким. Если слово встречается почти везде, например «и», «в», «на», его вес будет низким.

Проблема классических векторов: они большие и разреженные. Например, если словарь содержит 100 000 слов, каждый документ — это вектор из 100 000 координат.

Для уменьшения размерности используют **SVD**:

$$X \approx U_k \Sigma_k V_k^T$$

где X — матрица «документы × слова» или матрица совместной встречаемости, k — выбранная малая размерность.

После SVD слова или документы можно представить плотными векторами малой размерности.

Коротко для экзамена: TF-IDF делает важные редкие слова более значимыми, а SVD сжимает большие разреженные матрицы в компактные плотные векторы.

7. Word2Vec: модель CBOW

CBOW — Continuous Bag of Words. Эта модель Word2Vec предсказывает центральное слово по его контексту.

Схема:

$$(w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}) \rightarrow w_t$$

Например:

«Я люблю пить ___ утром»

Модель должна предсказать слово «кофе».

Контекстные слова переводятся в эмбединги, затем обычно усредняются:

$$v_{context} = \frac{1}{2m} \sum_{-m \leq j \leq m, j \neq 0} v_{t+j}$$

Затем через softmax предсказывается вероятность центрального слова:

$$P(w_t | context) = \frac{\exp(u_{w_t}^T v_{context})}{\sum_{w \in V} \exp(u_w^T v_{context})}$$

CBOW быстрее обучается и хорошо работает для частых слов. В лекции он сравнивается со Skip-Gram: CBOW предсказывает целевое слово по контексту, быстрее обучается и лучше подходит для больших корпусов.

Коротко для экзамена: CBOW берёт окружающие слова и по ним предсказывает центральное слово. Это быстрый способ обучить эмбединги.

8. Word2Vec: модель Skip-Gram

Skip-Gram работает наоборот: по центральному слову предсказывает слова контекста.

Схема:

$$w_t \rightarrow (w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2})$$

Цель модели — максимизировать вероятность контекстных слов:

$$P(context(w_t) | w_t) = \prod_{-m \leq j \leq m, j \neq 0} P(w_{t+j} | w_t)$$

Вероятность через softmax:

$$P(w_o | w_i) = \frac{\exp(u_{w_o}^T v_{w_i})}{\sum_{w \in V} \exp(u_w^T v_{w_i})}$$

где v_{w_i} — вектор входного слова, u_{w_o} — вектор выходного слова.

Проблема полного softmax: нужно суммировать по всему словарю, что дорого. Поэтому используют **negative sampling**: модель учится отличать реальные пары «слово-контекст» от случайных отрицательных пар.

Skip-Gram обычно лучше работает с редкими словами и качественнее улавливает семантику, но обучается медленнее, чем CBOW.

Коротко для экзамена: Skip-Gram берёт слово и учится предсказывать его соседей. Он медленнее CBOW, но часто даёт более качественные эмбединги, особенно для редких слов.

9. Слой nn.Embedding в PyTorch

nn.Embedding — это обучаемая матрица эмбедингов. Она хранит вектор для каждого токена словаря.

Если словарь размера V , а размер эмбединга d , то матрица имеет размер:

$$E \in \mathbb{R}^{V \times d}$$

Каждое слово сначала заменяется на индекс:

$$\text{"кошка"} \rightarrow 15$$

Потом nn.Embedding достаёт строку матрицы:

$$E[15] = v_{\text{кошка}}$$

Пример:

```
import torch
import torch.nn as nn

embedding = nn.Embedding(num_embeddings=10000, embedding_dim=300)

tokens = torch.tensor([15, 27, 391])
vectors = embedding(tokens)

print(vectors.shape) # torch.Size([3, 300])
```

Применение:

индексы слов → эмбединги → RNN/LSTM/GRU/Transformer

Во время обучения веса nn.Embedding обновляются градиентным спуском. То есть модель сама учит векторные представления слов под конкретную задачу.

Коротко для экзамена: nn.Embedding — это таблица, где каждому слову соответствует обучаемый плотный вектор. Она заменяет one-hot на компактные эмбединги.

10. Предобученные эмбединги. FastText

Предобученные эмбединги — это векторы слов, заранее обученные на большом корпусе. Их можно использовать в своей задаче вместо обучения с нуля.

Преимущество: модель уже знает общие семантические связи. Например:

$$\textit{king} - \textit{man} + \textit{woman} \approx \textit{queen}$$

Пример использования:

```
from gensim.models import KeyedVectors

model = KeyedVectors.load_word2vec_format("cc.ru.300.vec", binary=False)

print(model.most_similar("кошка"))
print(model.similarity("кошка", "собака"))
```

FastText отличается от Word2Vec тем, что представляет слово через символьные n-граммы.

Например, слово «кошка» можно разбить на части:

кош, ошк, шка

Итоговый вектор слова строится из векторов его подслов.

Преимущества FastText:

- лучше работает с морфологически богатыми языками;
- может строить векторы для редких и неизвестных слов;
- устойчивее к опечаткам;
- компактнее и гибче для разных форм слова.

Коротко для экзамена: предобученные эмбединги дают готовые векторы слов, а FastText улучшает Word2Vec тем, что учитывает внутреннюю структуру слова через символьные n-граммы.

11. Рекуррентная нейронная сеть, обучение и сложности применения

RNN — рекуррентная нейронная сеть, предназначенная для обработки последовательностей: текста, временных рядов, речи.

Главная идея: сеть хранит **скрытое состояние** h_t , которое несёт информацию о предыдущих элементах.

Формула базовой RNN:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b_h)$$

$$y_t = W_y h_t + b_y$$

где:

- x_t — вход на шаге t ;
- h_t — скрытое состояние;
- h_{t-1} — память о прошлом;
- y_t — выход;
- веса W_h, W_x, W_y общие для всех временных шагов.

Обучение RNN выполняется через **Backpropagation Through Time**. Сеть разворачивается во времени как глубокая сеть:

$$x_1 \rightarrow h_1 \rightarrow h_2 \rightarrow \dots \rightarrow h_T$$

После этого ошибка распространяется назад через все шаги.

Основные сложности:

1. **Vanishing gradient** — градиент затухает, и сеть плохо запоминает далёкое прошлое.
2. **Exploding gradient** — градиент становится слишком большим.
3. **Ограниченный контекст** — вся информация сжимается в один вектор.
4. **Последовательная обработка** — токены нельзя эффективно обрабатывать параллельно.
5. **Проблемы с длинными текстами** — модель может забывать начало последовательности.

Коротко для экзамена: RNN обрабатывает текст слева направо и хранит память в скрытом состоянии, но плохо работает с длинными зависимостями из-за затухания/взрыва градиентов.

12. Модуль LSTM

LSTM — Long Short-Term Memory. Это улучшенная RNN, созданная для хранения долгосрочной информации.

В LSTM есть отдельное состояние памяти:

$$c_t$$

и три основных гейта:

1. **Forget gate** — что забыть.
2. **Input gate** — что записать.
3. **Output gate** — что выдать наружу.

Формулы LSTM:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

Смысл:

- f_t решает, какую старую информацию сохранить;
- i_t решает, какую новую информацию записать;
- o_t решает, какая часть памяти попадёт в скрытое состояние.

Коротко для экзамена: LSTM решает проблему долгой памяти за счёт отдельной ячейки памяти c_t и гейтов, которые управляют забыванием, записью и чтением информации.

13. Модуль GRU

GRU — Gated Recurrent Unit. Это упрощённая альтернатива LSTM.

GRU объединяет идеи скрытого состояния и памяти. В отличие от LSTM, у GRU нет отдельного состояния c_t . Основная память хранится в h_t .

Главные гейты:

1. **Update gate** z_t — сколько старой информации сохранить.
2. **Reset gate** r_t — сколько прошлого учитывать при создании нового состояния.

Формулы:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

GRU обычно имеет меньше параметров, чем LSTM, поэтому быстрее обучается и требует меньше ресурсов.

Коротко для экзамена: GRU — более простая версия LSTM: меньше гейтов, нет отдельной памяти c_t , но всё равно лучше обычной RNN работает с долгосрочными зависимостями.

14. torch.nn.LSTM и torch.nn.GRU в PyTorch

В PyTorch LSTM и GRU реализованы как готовые классы.

Пример LSTM:

```
import torch
import torch.nn as nn

lstm = nn.LSTM(
    input_size=300,
    hidden_size=128,
    num_layers=2,
    batch_first=True,
    bidirectional=True
)

x = torch.randn(32, 50, 300) # batch=32, seq_len=50, embedding_dim=300

output, (h_n, c_n) = lstm(x)

print(output.shape) # [32, 50, 256], потому что bidirectional=True
```

Пример GRU:

```
gru = nn.GRU(
    input_size=300,
    hidden_size=128,
    num_layers=2,
    batch_first=True,
    bidirectional=False
)

output, h_n = gru(x)
```

Основные параметры:

Параметр	Значение
input_size	размер входного вектора, например embedding_dim
hidden_size	размер скрытого состояния
num_layers	число рекуррентных слоёв
batch_first=True	вход имеет форму [batch, seq_len, features]
bidirectional=True	используется двунаправленная сеть
dropout	регуляризация между слоями

Разница в выходах:

Для LSTM:

$$output, (h_n, c_n)$$

Для GRU:

$$output, h_n$$

Потому что у LSTM есть отдельная память c_t , а у GRU её нет.

Коротко для экзамена: в PyTorch LSTM возвращает output, h_n и c_n, а GRU возвращает только output и h_n. Оба модуля удобно использовать после слоя nn.Embedding.

15. Продвинутые RNN-архитектуры: BiLSTM, seq2seq для машинного перевода

BiLSTM — двунаправленная LSTM. Она читает последовательность в двух направлениях:

$$\vec{h}_t = LSTM_{forward}(x_1, \dots, x_t)$$

$$\overleftarrow{h}_t = LSTM_{backward}(x_T, \dots, x_t)$$

Итоговое представление:

$$h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t]$$

То есть модель учитывает контекст слева и справа. Это полезно для задач, где всё предложение известно заранее: классификация текста, NER, POS-tagging.

Ограничение: BiLSTM нельзя напрямую использовать для генерации текста слева направо, потому что при генерации будущие слова ещё неизвестны.

Seq2seq — архитектура «последовательность в последовательность». Она состоит из двух RNN:

$$Encoder \rightarrow Decoder$$

Схема:

$$x_1, x_2, \dots, x_T \rightarrow Encoder \rightarrow h_T \rightarrow Decoder \rightarrow y_1, y_2, \dots, y_M$$

В машинном переводе encoder читает предложение на исходном языке, сжимает его в скрытое состояние, а decoder генерирует перевод.

Проблемы базового seq2seq:

- всё предложение сжимается в один вектор;
- трудно переводить длинные предложения;
- расстояние между словом в encoder и его переводом в decoder может быть большим;
- длины исходного и целевого предложения часто различаются.

Улучшения:

1. **BiRNN / BiLSTM encoder** — учитывает контекст с двух сторон.
2. **Многослойные RNN** — несколько уровней обработки.
3. **Residual connections** — помогают передавать информацию и бороться с затуханием.
4. **Attention** — decoder смотрит не только на последнее состояние encoder, а на все скрытые состояния:

$$context_t = \sum_i \alpha_{t,i} h_i$$

где $\alpha_{t,i}$ — вес внимания к i -му слову исходного предложения.

Коротко для экзамена: BiLSTM читает текст в двух направлениях, а seq2seq использует encoder и decoder для задач вроде перевода. Главная проблема базового seq2seq — сжатие всего предложения в один вектор, поэтому добавляют attention.

Ниже — продолжение по вопросам **16-30** в формате краткого устного ответа. Основные источники: лекция 04 про Attention, Transformer и BERT ; лекция 05 про чат-боты ; лекция 06 про GPT, LLM и RLHF ; лекция 07 про продвинутые техники LLM, prompt engineering, LoRA, RAG и агентов .

16. Механизм Attention. Пример использования Attention

Attention — это механизм, который позволяет модели при обработке текущего токена «смотреть» не только на последнее скрытое состояние, как в RNN, а на разные части входной последовательности. В лекции подчёркивается, что attention помогает находить связи между словами на большом расстоянии и использовать любое доступное скрытое состояние, а не только последнее .

Основная формула:

$$Attention(Q, K, V) = softmax \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

где:

- Q — query, то есть «что ищем»;
- K — key, то есть «где ищем»;
- V — value, то есть «что извлекаем».

Пример: в машинном переводе при генерации слова перевода decoder может смотреть на разные слова исходного предложения. Например, при переводе фразы «I am a person» decoder для слова *personne* должен сильнее обратить внимание на слово *person*.

Коротко для экзамена: attention — это способ вычислить, какие части входа важны для текущего шага. Он особенно полезен в переводе и длинных текстах, потому что модель не обязана сжимать всё предложение в один вектор.

17. Transformer: общая архитектура, self-attention и multi-head attention

Transformer — архитектура, предложенная в статье *Attention is All You Need*. В отличие от RNN, она не обрабатывает токены строго последовательно, а использует attention и поэтому хорошо параллелится. В лекции указаны основные компоненты: encoder, decoder, position-wise feed-forward, layer normalization; преимущества — путь ($O(1)$) между позициями, параллельные вычисления и лучшее сохранение дальних зависимостей.

Self-attention работает внутри одной последовательности: каждое слово сопоставляется со всеми словами в предложении, для каждой пары считается вес связи, а итоговое представление слова получается как взвешенная сумма представлений всех слов.

Multi-head attention — это несколько attention-механизмов, работающих параллельно. Каждая «голова» учится находить свой тип связи: синтаксические связи, семантические отношения, кореференцию, позиционную информацию. Затем результаты голов объединяются:

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O$$

В лекции описан процесс: входной вектор проецируется в разные подпространства, attention считается параллельно, результаты конкатенируются, затем применяется финальное линейное преобразование.

Коротко для экзамена: self-attention позволяет каждому слову учитывать все остальные слова, а multi-head attention делает это сразу несколькими способами, чтобы ловить разные типы языковых связей.

18. Transformer: декодер, связь с энкодером, позиционное кодирование

Transformer состоит из **энкодера** и **декодера**. Энкодер обрабатывает входную последовательность, например предложение на исходном языке. Декодер генерирует выходную последовательность, например перевод.

В декодере есть две важные части:

1. **Masked self-attention** — декодер смотрит только на уже сгенерированные токены, чтобы не «подглядывать» в будущее.

2. **Cross-attention** — декодер обращается к выходам энкодера.

В лекции отдельно отмечено: в encoder-decoder cross-attention запросы Q идут из декодера, а ключи K и значения V — из энкодера; каждый токен декодера может смотреть на все токены энкодера, и это применяется на всех слоях декодера.

Формально:

$$\text{CrossAttention}(Q_{dec}, K_{enc}, V_{enc})$$

Позиционное кодирование нужно потому, что self-attention сам по себе не знает порядок слов. Для модели фразы «кошка поймала мышку» и «мышка поймала кошку» были бы слишком похожи без информации о позиции. Поэтому к эмбедингу токена добавляется позиционный вектор.

Формулы синусоидального позиционного кодирования:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$
$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Коротко для экзамена: энкодер понимает вход, декодер генерирует выход. Декодер использует masked self-attention для уже сгенерированных слов и cross-attention для связи с энкодером. Позиционное кодирование добавляет информацию о порядке слов.

19. Модель BERT

BERT — Bidirectional Encoder Representations from Transformers. Это модель на основе **только encoder-части Transformer**. В лекции указано: BERT Base имеет 12 слоёв, 768 hidden, 12 attention heads; BERT Large — 24 слоя, 1024 hidden, 16 heads. Главные идеи: двунаправленный контекст, масштабное предобучение и схема «pre-training + fine-tuning».

BERT отличается от GPT тем, что он не генерирует текст слева направо, а смотрит сразу на всё предложение. Поэтому он особенно хорош для задач понимания текста: классификация, NER, поиск ответа в тексте.

BERT обучается на двух задачах:

1. **Masked Language Modeling** — часть токенов скрывается, модель должна восстановить пропущенные слова:

$$L_{MLM} = - \sum_{i \in M} \log P(w_i | x_{\setminus M})$$

2. **Next Sentence Prediction** — бинарная классификация: является ли второе предложение продолжением первого. В лекции указано, что NSP учит модель различать связь между предложениями.

Вход BERT состоит из суммы трёх эмбедингов:

$$\text{Embedding} = \text{TokenEmbedding} + \text{SegmentEmbedding} + \text{PositionEmbedding}$$

Коротко для экзамена: BERT — это encoder-only Transformer для понимания текста. Он предобучается на MLM и NSP, а затем донастраивается под конкретные NLP-задачи.

20. Основные архитектуры чат-ботов и общая логика построения

В лекции выделяются основные типы архитектур чат-ботов:

Тип	Суть
Rule-based	Работают по заранее заданным правилам и дереву диалогов
ML-based	Обучаются на данных и используют NLP
Hybrid	Комбинируют правила и ML
AI-powered assistants	Построены на LLM и умеют генерировать ответы

Общая логика чат-бота строится вокруг двух понятий:

$$Intent + Entity \rightarrow Action$$

Intent — намерение пользователя. Например: «заказать пиццу», «узнать погоду», «забронировать столик».

Entity — сущности, важные для выполнения намерения. Например: город, дата, количество людей, тип товара.

Высокоуровневая архитектура:

1. **NLU** — понимает текст: определяет интент и сущности.
2. **Dialog Manager** — управляет состоянием диалога.
3. **Task Manager** — выполняет задачу во внешней системе.
4. **NLG** — формирует ответ пользователю.

Коротко для экзамена: чат-бот получает сообщение, определяет интент и сущности, обновляет состояние диалога, выбирает действие и генерирует ответ.

21. Классификация интентов. Машина состояний в чат-ботах

Классификация интентов — задача определить, что хочет пользователь. Например:

«Хочу заказать большую пиццу с грибами»

Интент:

$$intent = order_{pizza}$$

Для решения задачи собирают обучающую выборку запросов, размечают реальные диалоги, группируют похожие формулировки и обучают классификатор.

Формально:

$$\hat{y} = \arg \max_y P(y | x)$$

где x — текст пользователя, y — интент.

Машина состояний — это модель, где бот находится в одном из состояний и переходит между ними по событиям.

Пример для заказа пиццы:

$$welcome \rightarrow size_{election} \rightarrow toppings_{election} \rightarrow confirmation \rightarrow payment$$

Пример логики:

```

if state == "welcome" and user_intent == "order_pizza":
    state = "size_selection"
    bot.say("Какой размер пиццы выбрать?")

elif state == "size_selection":
    slots["size"] = user_text
    state = "toppings_selection"
    bot.say("Какие добавки хотите?")

```

Коротко для экзамена: классификация интенгов определяет цель пользователя, а машина состояний помогает управлять последовательностью шагов диалога.

22. NER и заполнение слотов в чат-ботах

NER — Named Entity Recognition, задача извлечения именованных сущностей из текста. В чат-ботах обычно извлекают даты, время, суммы, номера документов, имена, адреса и локации.

Пример:

«Забронируй столик завтра в 19:00 на двоих»

Сущности:

date = завтра, *time* = 19 : 00, *people* = 2

Slot filling — это заполнение заранее заданных параметров, нужных для выполнения задачи. Например, для бронирования нужны слоты:

restaurant, date, time, people

Если слот не заполнен, бот задаёт уточняющий вопрос:

«На какое время забронировать?»

Основные задачи slot filling:

- определить нужные слоты;
- извлечь значения из текста;
- проверить корректность;
- уточнить недостающие данные.

Коротко для экзамена: NER извлекает сущности из сообщения, а slot filling раскладывает эти сущности по нужным параметрам задачи.

23. Goal-oriented chatbot и обучение с подкреплением

Goal-oriented chatbot — это чат-бот, который помогает пользователю достичь конкретной цели: оформить заказ, забронировать билет, записаться к врачу. В лекции такие боты описаны как системы, которые понимают запросы своей сферы, отслеживают состояние диалога и выбирают действия для достижения конечной цели .

Архитектура goal-oriented chatbot:

$User \rightarrow NLU \rightarrow DialogStateTracker \rightarrow Policy \rightarrow Action \rightarrow NLG$

Где:

- **NLU** определяет интенг и сущности;
- **Dialog State Tracker** хранит состояние диалога;
- **Policy** выбирает следующее действие;
- **NLG** формирует ответ.

Обучение с подкреплением можно использовать для обучения **dialog policy**. Диалог представляется как MDP:

$$(S, A, R, P)$$

где:

- S — состояние диалога;
- A — действия бота;
- R — награда;
- P — вероятности переходов.

Пример награды:

$$R = \begin{cases} +20, & \text{за успешное выполнение задачи} \\ -1, & \text{за каждый лишний шаг} \\ -10, & \text{за провал диалога} \end{cases}$$

Коротко для экзамена: goal-oriented chatbot ведёт пользователя к конкретной цели, а reinforcement learning помогает обучить стратегию диалога так, чтобы задача решалась успешно и за меньшее число шагов.

24. Fine-tuning BERT и использование BERT для NLP-задач

Fine-tuning BERT — это дообучение уже предобученной модели BERT на конкретной задаче. Основная идея: BERT уже выучил хорошие языковые представления, поэтому для новой задачи достаточно добавить небольшой выходной слой и дообучить модель.

Примеры задач:

1. **Классификация текста** — берём embedding токена [CLS] и подаём в классификатор:

$$\hat{y} = \text{softmax}(Wh_{CLS} + b)$$

2. **NER** — классификация каждого токена:

$$\hat{y}_t = \text{softmax}(Wh_t + b)$$

3. **Question Answering** — модель предсказывает начало и конец ответа в тексте:

$$P_{start} = \text{softmax}(W_s h_t)$$

$$P_{end} = \text{softmax}(W_e h_t)$$

4. **Sentence pair classification** — например, определение, являются ли предложения перефразами.

Коротко для экзамена: fine-tuning BERT — это дообучение предобученного encoder-трансформера под конкретную задачу: классификацию, NER, поиск ответа, анализ пар предложений.

25. Pre-training, up-stream и down-stream задачи. Pre-training vs fine-tuning

Pre-training — предварительное обучение модели на большой общей задаче. Например, BERT обучается на masked language modeling и next sentence prediction.

Fine-tuning — дообучение этой модели на конкретной прикладной задаче.

В лекции идея объясняется так: сначала модель обучают на большом наборе данных для задачи X , а затем донастраивают на задачу Y ; предполагается, что X связана с Y , поэтому модель получает полезные представления.

Up-stream task — большая предварительная задача, на которой модель учится общему пониманию языка.

Пример:

MLM, NSP, next token prediction

Down-stream task — конечная прикладная задача.

Пример:

classification, NER, QA, sentiment analysis

Сравнение:

Подход	Что делаем	Пример
Pre-training	Учим модель на огромном корпусе	BERT восстанавливает masked tokens
Fine-tuning	Донастраиваем под задачу	BERT классифицирует отзывы

Коротко для экзамена: pre-training даёт модели общее знание языка, а fine-tuning адаптирует это знание под конкретную задачу.

26. Модель GPT. Архитектура и сравнение с BERT

GPT — Generative Pre-trained Transformer. Это модель на основе **decoder-only Transformer**. В лекции указано, что GPT использует авторегрессионный transformer decoder, работает слева направо и обучается как языковая модель.

Цель обучения GPT:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_{<t})$$

Функция потерь:

$$L = - \sum_{t=1}^T \log P(x_t | x_{<t})$$

Сравнение BERT и GPT:

Критерий	BERT	GPT
Архитектура	Encoder-only	Decoder-only
Контекст	Двунаправленный	Левый контекст
Главная задача	Понимание текста	Генерация текста
Pre-training	MLM + NSP	Next token prediction
Генерация текста	Не предназначен	Да
Примеры задач	NER, классификация, QA	чат-боты, генерация, продолжение текста

Коротко для экзамена: BERT лучше для понимания текста, потому что смотрит на всё предложение, а GPT лучше для генерации, потому что последовательно предсказывает следующий токен.

27. LLM: история развития, задачи, особенности. Обучение ChatGPT

LLM — большие языковые модели, обычно построенные на Transformer и обученные на огромных текстовых корпусах. В лекции отмечается рост размера моделей: от сотен миллионов параметров у BERT до сотен миллиардов и триллионных масштабов у современных LLM.

Уникальные особенности LLM:

- обучение по контексту без изменения весов;
- zero-shot и few-shot способности;
- генерация текста;
- решение задач, на которых модель явно не обучалась;
- способность следовать инструкциям;
- использование в чат-ботах, поиске, программировании, анализе текстов.

Процедура обучения ChatGPT-подобных моделей:

1. **Pre-training** — self-supervised обучение на огромных корпусах, обычно next token prediction.
2. **SFT** — supervised fine-tuning на парах «инструкция — хороший ответ».
3. **Reward Model** — обучение модели вознаграждения на человеческих предпочтениях.
4. **RLHF + PPO** — дообучение модели так, чтобы она давала ответы, предпочитаемые людьми.
5. **Оценка и безопасность** — фильтрация вредных ответов, тестирование, настройка поведения.

Коротко для экзамена: LLM — это крупные Transformer-модели, которые после pre-training умеют обобщать, обучаться по контексту и решать широкий спектр задач. ChatGPT обучается через pre-training, SFT и RLHF.

28. Pre-training LLM, датасеты и оценка качества LLM

Pre-training LLM — это обучение модели предсказывать следующий токен на огромных текстовых корпусах:

$$L = - \sum_{t=1}^T \log P(x_t | x_{<t})$$

В лекции перечислены типичные источники данных: Pile-CC, PubMed Central, Books3, OpenWeb-Text2, ArXiv, GitHub, FreeLaw, Stack Exchange, Wikipedia, PubMed Abstracts, Project Gutenberg, OpenSubtitles, EuroParl, YouTube Subtitles и другие. Также указано, что для базового обучения GPT-3 использовалось более 570 ГБ текстовых данных.

Оценивать LLM сложно, потому что нет одного универсального критерия «хорошего ответа»: разные задачи требуют разных метрик, а ответы могут быть правдоподобными, но ошибочными.

Методика 1 — MMLU. MMLU оценивает знания модели в разных областях: гуманитарные науки, естественные науки, математика, программирование. Формат — multiple choice с 4 вариантами ответа. Метрика — ассигасу. В лекции указано, что MMLU включает 57 задач разных областей и проверяет не только язык, но и предметные знания.

Методика 2 — Chatbot Arena / Arena Elo. Двум моделям дают один и тот же промпт, пользователь выбирает лучший ответ или ничью. Рейтинг пересчитывается по Elo:

$$R_{new} = R_{old} + K(S - E)$$

где S — фактический результат, E — ожидаемый результат.

Методика 3 — GSM8K / математические задачи. Проверяет способность модели решать школьные математические задачи с рассуждением. Метрика — ассигасу по правильному числовому ответу.

Методика 4 — HumanEval. Используется для оценки программирования: модель должна написать функцию по описанию. Метрика часто называется `pass@k` — прошёл ли хотя бы один из k сгенерированных вариантов тесты.

Коротко для экзамена: LLM предобучают на задаче next token prediction на огромных корпусах. Качество оценивают через бенчмарки: MMLU для знаний, Arena Elo для предпочтений людей, GSM8K для рассуждений, HumanEval для кода.

29. RLHF и PPO для LLM

RLHF — Reinforcement Learning from Human Feedback, обучение с подкреплением на основе человеческой обратной связи. В лекции указано, что RLHF нужен потому, что обычный fine-tuning на статических данных не охватывает все человеческие ценности, нормы и субъективные критерии качества ответа.

Этапы RLHF:

1. Собирают промпты.
2. Модель генерирует несколько ответов.
3. Люди выбирают лучший ответ.
4. На этих предпочтениях обучается **Reward Model**.
5. LLM дообучается так, чтобы максимизировать reward.

Reward Model можно записать так:

$$r_{\phi}(x, y)$$

где x — промпт, y — ответ модели.

PPO — Proximal Policy Optimization. Это алгоритм RL, который улучшает политику модели, но не позволяет ей слишком резко отойти от исходной модели. Это важно, чтобы модель не «сломалась» и не начала оптимизировать reward странными способами.

Упрощённо цель PPO:

$$\max_{\theta} \mathbb{E}[r_{\phi}(x, y)] - \beta KL(\pi_{\theta} || \pi_{ref})$$

где:

- π_{θ} — обучаемая модель;
- π_{ref} — исходная модель;
- (KL) ограничивает отклонение от исходного поведения;
- β регулирует силу штрафа.

Коротко для экзамена: RLHF учит LLM отвечать так, как предпочитают люди. PPO используется для стабильной донастройки модели по reward model, не давая ей слишком далеко уйти от исходной языковой модели.

30. Prompt engineering: суть и принципы качественного промпта

Prompt engineering — это задача правильного составления запроса к LLM. В лекции указано, что качество ответа зависит от ясности запроса, полноты контекста, формулировок, примеров, языка, структуры разделов промпта, длины входа и методики общения с моделью.

Хороший промпт обычно содержит:

1. **Роль** — кем должна быть модель.
2. **Контекст** — исходные данные и ситуация.
3. **Задачу** — что нужно сделать.
4. **Ограничения** — стиль, объём, запреты.

5. **Формат ответа** — таблица, список, код, план.
6. **Примеры** — если нужно показать шаблон.

Пример структуры CO-STAR из лекции:

Компонент	Смысл
C — Context	Контекст
O — Objective	Цель
S — Style	Стиль
T — Tone	Тон
A — Audience	Аудитория
R — Response Format	Формат ответа

Для сложных задач можно использовать chain-of-thought prompting: просить модель разбить решение на шаги. В лекции отмечено, что это повышает точность, облегчает проверку логики и помогает выявлять ошибки .

Коротко для экзамена: prompt engineering — это грамотное проектирование запроса к LLM. Хороший промпт должен ясно задавать цель, контекст, ограничения и формат ответа.

Ниже — ответы на вопросы **31-40**. Основная база: лекции 06-07 по LLM, prompt engineering, CoT, адаптерам, LoRA, RAG, квантизации и LLM-агентам . Для пунктов про **MCP, JSON Schema, structured outputs, constrained decoding и OpenCode** добавлены актуальные официальные источники, потому что эти темы быстро обновляются.

31. In-context learning, few-shot, one-shot, zero-shot, Chain-of-Thought

In-context learning — это способность LLM решать новую задачу прямо из текста запроса, без изменения весов модели. То есть модель не дообучают градиентным спуском, а просто дают ей инструкцию и, при необходимости, несколько примеров в промпте.

В лекции это описано так: LLM могут обучаться по контексту на основе текстовой инструкции, иногда с несколькими правильными примерами, и решать новые задачи без дополнительного обучения весов .

Отличие от обычного ML/DL:

Подход	Что меняется
Classical ML	обучается отдельная модель под задачу
Fine-tuning	меняются веса нейросети
In-context learning	веса не меняются, меняется только промпт

Варианты:

zero-shot : инструкция без примеров

one-shot : инструкция + 1 пример

few-shot : инструкция + несколько примеров

Chain-of-Thought prompting — это просьба к модели рассуждать пошагово. Например: «Разберём решение по шагам». Такой подход повышает качество на сложных задачах, потому что модель сначала строит промежуточные рассуждения, а потом выдаёт ответ .

Коротко: in-context learning — это обучение «внутри промпта», без изменения весов. Zero-shot — без примеров, one-shot — один пример, few-shot — несколько примеров, CoT — пошаговое рассуждение.

32. Chain-of-Thought prompting, улучшенный CoT и размышляющие модели

Chain-of-Thought — метод prompting, при котором LLM побуждают строить решение по шагам. В лекции указано, что CoT повышает точность, облегчает проверку логики, делает решение прозрачнее и помогает находить ошибки.

Базовая схема:

$$Question \rightarrow Reasoning\ steps \rightarrow Final\ answer$$

Пример промпта:

Давай рассуждать пошагово. Сначала выдели данные, затем составь план, затем реши задачу.

Улучшенные варианты:

Self-Consistency CoT — модель генерирует несколько цепочек рассуждений, а затем выбирается наиболее согласованный ответ.

$$answer = majority_vote(CoT_1, CoT_2, \dots, CoT_k)$$

Tree-of-Thought — рассуждение строится не одной цепочкой, а деревом вариантов. Модель может рассматривать несколько путей решения и выбирать лучший.

Graph-of-Thought — более общий вариант: рассуждения могут быть графом, где идеи связаны не только линейно или древовидно.

Размышляющие модели — модели, которые специально обучаются оценивать не только финальный ответ, но и промежуточные шаги. В лекции упоминается подход PRM: Process Reward Model, где награда даётся за правильность каждого шага рассуждения, а не только за конечный результат.

Коротко: обычный CoT — одна цепочка рассуждений; улучшенный CoT строит несколько вариантов, дерево или граф рассуждений; размышляющие модели обучаются проверять и улучшать сам процесс рассуждения.

33. Parameter-efficient fine-tuning: адаптеры

Parameter-efficient fine-tuning — это дообучение LLM, при котором обучается не вся модель, а только небольшой набор дополнительных параметров. Это дешевле, быстрее и снижает риск «сломать» базовые знания модели.

Адаптер — небольшой блок, который добавляется внутрь Transformer-блока. Основная модель замораживается, а обучаются только параметры адаптера. В лекции адаптеры описаны как дополнения к LLM, добавляющие минимум параметров без изменения основной модели.

Типичная схема адаптера:

$$x \rightarrow W_{down} \rightarrow activation \rightarrow W_{up} \rightarrow +x$$

Формула:

$$Adapter(x) = x + W_{up}\sigma(W_{down}x)$$

где:

- W_{down} уменьшает размерность;
- W_{up} возвращает размерность обратно;
- x добавляется через residual connection.

В лекции пример: если вход имеет размерность 1024, а bottleneck — 24, то адаптер добавит:

$$1024 \cdot 24 + 24 \cdot 1024 = 49152$$

параметров, а обычный полносвязный слой 1024×1024 имел бы:

$$1024^2 = 1048576$$

параметров .

Коротко: адаптер — маленький обучаемый блок внутри Transformer. Базовая LLM заморожена, обучается только адаптер, поэтому дообучение становится намного дешевле.

34. LoRA: принцип работы, архитектура, эффективность и преимущества

LoRA — Low-Rank Adaptation. Это метод parameter-efficient fine-tuning, где вместо полного изменения матрицы весов обучается её низкоранговое обновление.

Обычный fine-tuning:

$$W_{ft} = W_{pt} + \Delta W$$

LoRA заменяет полное обновление ΔW произведением двух маленьких матриц:

$$\Delta W \approx AB$$

$$W_{ft} = W_{pt} + \alpha AB$$

где:

- W_{pt} — предобученные веса;
- A и B — обучаемые низкоранговые матрицы;
- r — ранг, причём $r \ll d$;
- α — коэффициент масштабирования.

Если исходная матрица:

$$W \in \mathbb{R}^{d \times d}$$

то LoRA использует:

$$A \in \mathbb{R}^{d \times r}, \quad B \in \mathbb{R}^{r \times d}$$

Количество обучаемых параметров:

$$2dr$$

вместо:

$$d^2$$

В лекции подчёркивается, что LoRA оставляет предобученные слои неизменными, а обучает только низкоранговое разложение. Также указано, что эффективность объясняется низкой внутренней размерностью больших моделей: не все параметры реально нужны для адаптации под задачу .

Преимущества LoRA:

- обучается мало параметров;
- меньше память и вычислительные затраты;
- базовая модель не меняется;
- можно хранить разные LoRA-адаптеры для разных задач;
- ниже риск катастрофического забывания;
- можно приблизиться к качеству полного fine-tuning.

Коротко: LoRA не меняет всю матрицу весов, а обучает маленькое низкоранговое обновление. Поэтому метод дешёвый, быстрый и хорошо подходит для дообучения больших LLM.

35. Квантизация, fp32/fp16/bfloat16, симметричная и асимметричная квантизация

Квантизация — это уменьшение точности представления чисел в модели. Например, веса были в float32, а после квантизации хранятся в int8. В лекции квантизация описана как перевод параметров модели из формата высокой точности в менее затратный формат, чтобы сократить память и ускорить инференс.

Форматы:

Формат	Биты	Смысл
fp32	32	высокая точность, много памяти
fp16	16	меньше память, быстрее на GPU
bfloat16	16	меньше мантисса, но широкий диапазон экспоненты как у fp32

Типичная линейная квантизация:

$$q = \text{round}\left(\frac{x}{s} + z\right)$$

$$\hat{x} = s(q - z)$$

где:

- x — исходное float-значение;
- q — квантованное целое значение;
- s — scale;
- z — zero-point;
- $\hat{x}$ — восстановленное приближённое значение.

Асимметричная квантизация точно отображает минимальное и максимальное значение исходного диапазона:

$$s = \frac{x_{max} - x_{min}}{q_{max} - q_{min}}$$

$$z = \text{round}\left(q_{min} - \frac{x_{min}}{s}\right)$$

Плюс: лучше использует диапазон. Минус: нужна дополнительная логика для zero-point.

Симметричная квантизация использует диапазон вокруг нуля:

$$s = \frac{\max(|x_{min}|, |x_{max}|)}{q_{max}}$$

$$q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right), q_{min}, q_{max}\right)$$

Плюс: проще и быстрее. Минус: если все значения положительные, например после ReLU, часть диапазона тратится зря; в лекции сказано, что в таком случае можно фактически потерять 1 бит .

Квантизация с насыщением означает, что значения за пределами выбранного диапазона обрезаются:

$$x_{clip} = \min(\max(x, \alpha), \beta)$$

Потом квантизация применяется уже к x_{clip} . В лекции указано, что цель — минимизировать потерю информации, а потерю можно измерять дивергенцией Кульбака-Лейблера .

Коротко: квантизация уменьшает точность чисел ради экономии памяти и ускорения. Симметричная проще, асимметричная точнее использует диапазон, saturation-квантизация обрезает выбросы.

36. Retrieval Augmented Generation и базовая архитектура RAG

RAG — Retrieval Augmented Generation, генерация с дополнением внешними найденными данными. Главная идея: LLM не отвечает только из своих весов, а сначала получает релевантные документы из внешней базы знаний.

В лекции RAG предлагается как способ бороться с галлюцинациями и незнанием фактов из приватных источников .

Базовая схема:

$$Documents \rightarrow Chunks \rightarrow Embeddings \rightarrow Vector\ index$$

Потом при запросе:

$$Query \rightarrow Query\ embedding \rightarrow TopK\ retrieval \rightarrow Prompt + Context \rightarrow LLM \rightarrow Answer$$

Формально:

$$e_i = Encoder(d_i)$$

$$e_q = Encoder(q)$$

$$TopK = \arg \max_i \cos(e_q, e_i)$$

Затем найденные фрагменты добавляются в промпт:

$$Prompt = Instruction + Question + Retrieved\ context$$

В лекции базовая реализация RAG описана так: текст разбивается на фрагменты, фрагменты переводятся в векторы, векторы помещаются в индекс, по запросу ищутся подсказки, а затем для LLM создаётся промпт с найденной фактической информацией .

Коротко: RAG = поиск + генерация. Сначала ищем релевантные документы, затем даём их LLM как контекст, чтобы ответ был более фактическим.

37. Структурированный вывод LLM для вызова инструментов: JSON, JSON Schema, Pydantic, XML-теги

Для вызова инструментов LLM должна выдавать не просто красивый текст, а **машиночитаемую команду**: какой инструмент вызвать и с какими аргументами.

Пример:

```
{
  "tool": "get_weather",
  "arguments": {
    "city": "Riga",
    "date": "today"
  }
}
```

JSON нужен как удобный формат обмена данными между LLM и программой.

JSON Schema задаёт правила структуры: какие поля должны быть, какие типы допустимы, какие значения обязательны. JSON Schema официально описывает структуру и правила JSON-данных, а валидатор проверяет, соответствует ли JSON этой схеме ([JSON Schema](#)).

Пример схемы:

```
{
  "type": "object",
  "properties": {
    "tool": {"type": "string"},
    "arguments": {"type": "object"}
  },
  "required": ["tool", "arguments"]
}
```

Pydantic используется в Python как удобный способ описывать такие структуры классами и валидировать данные. Pydantic основан на type hints, выполняет валидацию и может генерировать JSON Schema из моделей ([pydantic.dev](#)).

Пример:

```
from pydantic import BaseModel

class WeatherArgs(BaseModel):
    city: str
    date: str
```

XML-теги часто используют в промптах для разделения частей:

```
<question>Какая погода в Риге?</question>
<tool_call>{"tool": "get_weather", "arguments": {"city": "Riga"}}</tool_call>
```

Structured Outputs нужны, чтобы модель строго следовала схеме. Например, OpenAI описывает Structured Outputs как механизм, при котором ответы модели надёжно соответствуют JSON Schema, заданной разработчиком ([OpenAI](#)).

Коротко: JSON — формат команды, JSON Schema — контракт структуры, Pydantic — валидация в Python, XML-теги — разметка частей промпта и ответа.

38. Model Context Protocol: цель, архитектура, сущности сервера и транспорт

MCP — Model Context Protocol. Его цель — стандартизировать подключение LLM-приложений к внешним данным и инструментам. Официальная документация описывает MCP как клиент-серверную архитектуру, где MCP host подключается к одному или нескольким MCP servers через MCP clients ([modelcontextprotocol.io](#)).

Участники:

MCP Host → MCP Client → MCP Server

- **Host** — AI-приложение, например IDE или чат-интерфейс.
- **Client** — компонент, который держит соединение с сервером.
- **Server** — программа, которая предоставляет контекст или инструменты.

MCP состоит из двух слоёв:

1. **Data layer** — JSON-RPC-протокол, lifecycle, tools, resources, prompts, notifications.
2. **Transport layer** — механизм передачи сообщений между клиентом и сервером (modelcontextprotocol.io).

Основные сущности сервера:

Сущность	Назначение
Tools	функции, которые можно вызвать
Resources	данные и контекст
Prompts	шаблоны промптов

В документации MCP прямо указано, что серверы могут предоставлять tools, resources и prompts; tools — это исполняемые функции, resources — источники данных, prompts — переиспользуемые шаблоны (modelcontextprotocol.io).

Транспортный уровень:

- **stdio** — общение через stdin/stdout, обычно для локальных серверов;
- **Streamable HTTP** — удалённый доступ по HTTP, с возможной потоковой передачей (modelcontextprotocol.io).

Коротко: MCP — это стандартный способ подключать LLM к инструментам и данным. Host создаёт client, client общается с server, server отдаёт tools/resources/prompts через JSON-RPC.

39. Валидация вывода и constrained decoding

Проблема: LLM может выдать почти правильный JSON, но с ошибкой: пропущена скобка, неправильный тип поля, нет обязательного аргумента. Для tool calling это критично, потому что программа не сможет вызвать инструмент.

Есть два подхода.

1. Валидация после генерации

Сначала модель генерирует ответ, затем программа проверяет его:

LLM output → Parser → Validator

Если ошибка — просим модель исправить или повторяем генерацию.

Плюсы:

- просто реализовать;
- работает с любыми моделями;
- можно использовать Pydantic/JSON Schema.

Минусы:

- не гарантирует правильный результат с первого раза;
- нужны ретраи;
- модель может снова ошибиться.

2. Constrained decoding

Здесь ограничения применяются прямо во время генерации. На каждом шаге запрещаются токены, которые нарушили бы схему или грамматику.

$$P'(token) = \begin{cases} P(token), & token \in AllowedTokens \\ 0, & token \notin AllowedTokens \end{cases}$$

Затем вероятности нормализуются.

Constrained decoding используется для structured outputs: например, OpenAI пишет, что Structured Outputs решают проблему соответствия схеме за счёт constraints под JSON Schema и обучения моделей лучше понимать схемы ([OpenAI](#)). В документации Claude также указано, что structured outputs гарантируют schema-compliant responses через constrained decoding, снижая ошибки парсинга, типов и обязательных полей ([Claude](#)).

Сравнение:

Критерий	Валидация	Constrained decoding
Когда применяется	после генерации	во время генерации
Гарантия синтаксиса	нет	обычно да
Простота	проще	сложнее
Ретраи	часто нужны	меньше нужны
Семантическая правильность	не гарантирует	тоже не гарантирует

Коротко: валидация проверяет ответ после генерации, а constrained decoding не даёт модели сгенерировать структурно неправильный ответ. Но оба подхода не гарантируют, что выбранный инструмент и аргументы будут смыслово правильными.

40. Архитектура многоагентной среды, преимущества, пример OpenCode

Многоагентная среда — это система, где задачу решает не один LLM-агент, а несколько специализированных агентов. В лекции LLM-агент определяется как автономная система на базе LLM, способная планировать действия и принимать решения; основные компоненты агента — LLM, планирование, память и инструменты .

Типичная архитектура:

$$User \rightarrow Coordinator \rightarrow Agent_1, Agent_2, \dots, Agent_n \rightarrow Tools/Memory$$

Например:

- **Planner** — строит план;
- **Researcher** — ищет информацию;
- **Coder** — пишет код;
- **Reviewer** — проверяет ошибки;
- **Executor** — запускает инструменты.

Преимущества:

- специализация агентов;
- параллельное выполнение частей задачи;
- лучшее разделение прав доступа;
- возможность проверки одним агентом работы другого;
- логирование и прозрачность действий;
- масштабируемость.

Пример — **OpenCode**. Официальная документация описывает OpenCode как open source AI coding agent, доступный как терминальный интерфейс, desktop app или IDE extension ([opencode.ai](#)). В OpenCode есть специализированные agents с разными промптами, моделями и доступом к инструментам; документация выделяет primary agents и subagents, а также встроенные агенты Build и Plan ([opencode.ai](#)).

Упрощённая схема OpenCode-подобной среды:

User request → Plan agent → Build agent → Review/Test agent

Коротко: многоагентная среда распределяет задачу между несколькими специализированными LLM-агентами. Это повышает управляемость, качество проверки и масштабируемость. OpenCode — пример среды для разработки, где можно использовать разных агентов для планирования, изменения кода и анализа.